

CSCI 4100

Operating Systems and Architecture

Chapter 1: Introduction

Topics

- 1.1 – What is an Operating System?
- 1.2 – Operating System Evaluation
- 1.3 – Operating Systems: Past, Present and Future

What Is an Operating System?

- An **operating system (OS)** plays three different roles on a computer.
 1. Referee: manages a computer's resources on behalf of its users and their applications.
 2. Illusionist: abstracts the physical hardware of the computer to provide a simplified virtual interface.
 3. Glue: provides a common set of services for applications to use.

An Operating System Is a Referee

- Sharing a computer's resources requires a significant amount of coordination.
- **Resource allocation** balances the needs of applications with global system performance.
- **Fault isolation** protects all applications from errors in some applications.
- Applications need to be able to communicate with each other without compromising fault isolation.

An Operating System Is an Illusionist.

- **Virtualization** provides applications with a simplified view of a computer's hardware.
- Each application runs as if it has exclusive access to one or more processors.
- **Virtual memory** provides the illusion of access to unlimited contiguous memory.
- The **file** abstraction provides a high-level interface to data stored persistently on storage devices.
- A **virtual machine** virtualizes the entire computer so that a guest OS can run on top of a host OS.

An Operating System Is Glue.

- Resource sharing and virtualization are simplified by providing a common set of system services.
- Standard file system and messaging interfaces make it easier for applications to share data.
- Generic interfaces simplify accessing services provided by a wide variety of hardware devices.
- A graphical user interface (GUI) library provides a common look and feel for applications.

Operating System Evaluation

- How do we evaluate the design of an operating system?
- Five criteria:
 - **Reliability and Availability** – does the operating system do what you want?
 - **Security** – can the operating system be corrupted by an attacker?
 - **Portability** – is the operating system easy to move to new hardware platforms?
 - **Performance** – is the user interface responsive, or does the operating system impose too much overhead?
 - **Adoption** – How many other users are there for this operating system?
- There will be tradeoffs between these criteria.

Reliability and Availability

- **Reliability** means that an OS does what it is designed to do.
 - An OS provides fault isolation for applications, but faults in the OS itself can cause serious problems.
 - Design errors can be used by attackers to gain unauthorized access to a system.
 - Even rarely used code must work correctly.
- **Availability** refers to the percentage of time that an OS is usable.
 - Reliability and availability are independent from one another.
 - **Mean time to failure (MTTF)** is a measure of how frequently failures occur.
 - **Mean time to repair (MTTR)** is the time required to restore a system after a failure.

Security

- **Security** means the operation of a computer can not be compromised by attackers.
 - No computer is perfectly secure, but an OS should be designed to minimize the chances of a successful attack.
 - Fault isolation can mitigate the effects of an attack.
 - Applications still need to be able to communicate and share data.
 - **Security policy** defines what operations are permitted.
 - **Enforcement mechanisms** ensure that only permitted actions are allowed.

Portability

- **Portability** means that an OS and its applications do not have to be rewritten from scratch to run on different hardware.
 - Most operating systems are built from existing codebases.
 - An **abstract virtual machine (AVM)** provides a standard interface for applications.
 - A **hardware abstraction layer (HAL)** allows the OS itself to be implemented independent of the hardware it is running on.

Performance

- **Performance** refers to the efficient management of the resources of an OS.
 - **Overhead** is the resource cost added by implementing an abstraction.
 - **Fairness** refers to how resources are allocated to different users or applications.
 - **Response time** is the amount of time a task takes to run, from start to completion.
 - **Throughput** is the rate at which the system completes tasks.
 - **Predictability** refers to the consistency of a system's response time.

Adoption

- The success of an OS also depends on how widely it is adopted by users, application developers, and hardware designers.
 - The **network effect** occurs when the number of people who adopt a technology affects its value.
 - A **proprietary system** is under the control of a single company (e.g. MacOS, Windows)
 - An **open system** has a publicly available codebase (e.g. Linux, Android)

Moore's Law and Early Operating Systems

- **Moore's Law** (1975 version): transistor density doubles every two years.
 - Exponential improvement in other areas as well.
 - Performance/capacity goes up while cost goes down.
 - Fundamental OS challenges are still (mostly) the same.
- Early operating systems were runtime libraries for large expensive computers.
 - Single-user systems.
 - Provided standard I/O routines.
 - Very inefficient.

Multi-User Operating Systems

- **Batch operating systems** support multiple users by operating on a queue of jobs.
 - Each job is loaded into memory, run, then unloaded.
 - **Direct memory access (DMA)** is used to load one job while another job is running.
 - When a job has finished loading, an **interrupt** is generated, and the processor pauses the program it is running to initiate the next load.
- **Multitasking** systems load multiple programs into memory and can switch to a new job if the current job is waiting on an I/O operation.
- Fault isolation is provided by added hardware memory protection.

Time-Sharing Operating Systems

- **Time-sharing operating systems** support interactive computing.
 - Each keystroke, mouse action, or other input event generates an interrupt.
 - An **interrupt handler** reads and puts the event in an event queue.
 - When control is returned to the application, it retrieves and processes the event.
 - **Personal computers** are single-user time-sharing systems.

Modern Operating Systems

- An OS for a personal computer must provide interactivity and reliability.
- A smartphone OS is focused on responsiveness and minimizing battery usage.
- A server OS coordinates multiple network connections and protects against attacks.
- A **virtual machine monitor** must minimize virtualization overhead.
- An embedded system runs a customized OS along with device specific software.
- Server clusters aggregate multiple servers to provide high availability and fault tolerance.

Future Operating Systems

- Very large-scale data centers.
- Very large-scale multicore systems.
- Ubiquitous portable computing devices.
- Very heterogeneous systems.
- Very large-scale storage.